

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)

Assessment Tool Weightage: 20%

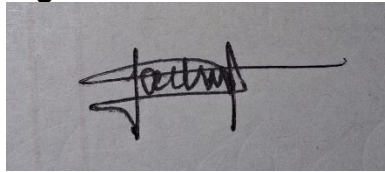
QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze
8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest	BL5 – Evaluate

	appropriate integrity constraints.	
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I J a s h w a n t h . G . R - 1 9 2 5 2 4 3 8 4 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. University Database – Normalize up to 3NF

Scenario Explanation

The university stores **student, course, and faculty information in a single table**. This causes data redundancy because faculty and course details are repeated for every student. Updating faculty information may require multiple changes, leading to update anomalies.

Solution

Normalize the database into separate tables.

QUERY

```
CREATE DATABASE UniversityDB;
```

```
USE UniversityDB;
```

```
CREATE TABLE Faculty(  
    FacultyID INT PRIMARY KEY,  
    FacultyName VARCHAR(50)  
);
```

```
CREATE TABLE Student(  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50)  
);
```

```
CREATE TABLE Course(  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50),  
    FacultyID INT,  
    FOREIGN KEY(FacultyID) REFERENCES Faculty(FacultyID)  
);
```

```
CREATE TABLE Enrollment(
    StudentID INT,
    CourseID INT,
    PRIMARY KEY(StudentID,CourseID),
    FOREIGN KEY(StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY(CourseID) REFERENCES Course(CourseID)
);
```

OUTPUT

```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds)
USE UniversityDB;
[ Edit inline ] [ Edit ] [ Create PHP code ]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds)
CREATE TABLE Faculty( FacultyID INT PRIMARY KEY, FacultyName VARCHAR(50) );
[ Edit inline ] [ Edit ] [ Create PHP code ]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0007 seconds)
CREATE TABLE Student( StudentID INT PRIMARY KEY, StudentName VARCHAR(50) );
[ Edit inline ] [ Edit ] [ Create PHP code ]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds)
CREATE TABLE Course( CourseID INT PRIMARY KEY, CourseName VARCHAR(50), FacultyID INT, FOREIGN KEY(FacultyID) REFERENCES Faculty(FacultyID) );
[ Edit inline ] [ Edit ] [ Create PHP code ]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0008 seconds)
CREATE TABLE Enrollment( StudentID INT, CourseID INT, PRIMARY KEY(StudentID,CourseID), FOREIGN KEY(StudentID) REFERENCES Student(StudentID), FOREIGN KEY(CourseID) REFERENCES Course(CourseID) );
```

Result: The schema is normalized to **3NF**, eliminating redundancy and anomalies.

Q2. Employees Working in Multiple Departments

Scenario Explanation

A company allows employees to work in multiple departments. Since one employee can belong to many departments, a separate mapping table is required. The task is to retrieve employees working in more than one department.

QUERY

```
CREATE DATABASE CompanyDB;
```

```
USE CompanyDB;
```

```
CREATE TABLE Employee(
```

```
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50)  
);
```

```
CREATE TABLE Department(  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50)  
);
```

```
CREATE TABLE EmployeeDepartment(  
    EmpID INT,  
    DeptID INT,  
    FOREIGN KEY(EmpID) REFERENCES Employee(EmpID),  
    FOREIGN KEY(DeptID) REFERENCES Department(DeptID)  
);
```

```
INSERT INTO Employee VALUES  
(1,'John'),  
(2,'Alice'),  
(3,'David');
```

```
INSERT INTO Department VALUES  
(101,'HR'),  
(102,'IT'),  
(103,'Finance');
```

```
INSERT INTO EmployeeDepartment VALUES  
(1,101),
```

(1,102),

(2,102),

(3,101),

(3,103);

SELECT e.EmpID,e.EmpName

FROM Employee e

JOIN EmployeeDepartment ed

ON e.EmpID=ed.EmpID

GROUP BY e.EmpID,e.EmpName

HAVING COUNT(ed.DeptID)>1;

OUTPUT

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

USE CompanyDB;

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

CREATE TABLE Employee(EmpID INT PRIMARY KEY, EmpName VARCHAR(50));

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

CREATE TABLE Department(DeptID INT PRIMARY KEY, DeptName VARCHAR(50));

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

CREATE TABLE EmployeeDepartment(EmpID INT, DeptID INT, FOREIGN KEY(EmpID) REFERENCES Employee(EmpID), FOREIGN KEY(DeptID) REFERENCES Department(DeptID));

[Edit inline] [Edit] [Create PHP code]

✓ 3 rows inserted. (Query took 0.0012 seconds.)

INSERT INTO Employee VALUES (1,'John'), (2,'Alice'), (3,'David');

[Edit inline] [Edit] [Create PHP code]

✓ 3 rows inserted. (Query took 0.0012 seconds.)

INSERT INTO Department VALUES (101,'HR'), (102,'IT'), (103,'Finance');

[Edit inline] [Edit] [Create PHP code]

✓ 5 rows inserted. (Query took 0.0014 seconds.)

INSERT INTO EmployeeDepartment VALUES (1,101), (1,102), (2,102), (3,101), (3,103);

[Edit inline] [Edit] [Create PHP code]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✓ Showing rows 0 - 1 (2 total, Query took 0.0005 seconds.)

SELECT e.EmpID,e.EmpName FROM Employee e JOIN EmployeeDepartment ed ON e.EmpID=ed.EmpID GROUP BY e.EmpID,e.EmpName HAVING COUNT(ed.DeptID)>1;

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

EmpID EmpName

1 John

3 David

Q3. Candidate Key and BCNF

Scenario Explanation

The relation contains multiple functional dependencies. We must identify the candidate key and verify whether the relation satisfies BCNF. If it does not, decompose it.

QUERY

```
CREATE TABLE R1(
```

```
    B INT PRIMARY KEY,
```

```
    C VARCHAR(30)
```

```
);
```

```
CREATE TABLE R2(
```

```
    A INT PRIMARY KEY,
```

```
    B INT,
```

```
    D VARCHAR(30),
```

```
    FOREIGN KEY(B) REFERENCES R1(B)
```

```
);
```

OUTPUT

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

```
CREATE TABLE R1( B INT PRIMARY KEY, C VARCHAR(30) );
```

[Edit inline](#)] [\[Edit \]](#) [\[Create PHP code \]](#)

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
CREATE TABLE R2( A INT PRIMARY KEY, B INT, D VARCHAR(30), FOREIGN KEY(B) REFERENCES R1(B) );
```

[Edit inline](#)] [\[Edit \]](#) [\[Create PHP code \]](#)

Result: The relation is decomposed into **BCNF**, reducing redundancy.

Q4. Banking System – Views and Constraints

Scenario Explanation

A bank should not allow every user to access all customer details. Views provide restricted access, while constraints ensure only valid data is stored.

QUERY

```
CREATE DATABASE BankDB;
```

```
USE BankDB;
```

```
CREATE TABLE Customer(  
    AccountNo INT PRIMARY KEY,  
    CustomerName VARCHAR(50) NOT NULL,  
    Balance DECIMAL(10,2) CHECK(Balance>=0),  
    Phone VARCHAR(15) UNIQUE  
);
```

```
CREATE VIEW Customer_View AS  
  
SELECT AccountNo, CustomerName, Balance  
  
FROM Customer;  
  
INSERT INTO Customer(AccountNo, CustomerName, Balance, Phone)  
  
VALUES  
  
(1001, 'Rahul Sharma', 25000.50, '9876543210'),  
(1002, 'Priya Singh', 45000.00, '9876543211'),  
(1003, 'Amit Kumar', 12000.75, '9876543212'),  
(1004, 'Sneha Reddy', 68000.25, '9876543213'),  
(1005, 'Arjun Patel', 35000.00, '9876543214');  
  
SELECT * FROM Customer_View;
```

OUTPUT

AccountNo	CustomerName	Balance
1001	Rahul Sharma	25000.50
1002	Priya Singh	45000.00
1003	Amit Kumar	12000.75
1004	Sneha Reddy	68000.25
1005	Arjun Patel	35000.00

Result: The view displays only required information while constraints maintain data integrity.

Q5. Sales Database Normalization

Scenario Explanation

The sales database stores customer and product details repeatedly in one table, causing redundancy and anomalies. The database should be normalized.

QUERY

```
CREATE TABLE Customer(
```

```
    CustomerID INT PRIMARY KEY,
```

```
    CustomerName VARCHAR(50)
```

```
);
```

```
CREATE TABLE Product(
```

```
    ProductID INT PRIMARY KEY,
```

```
    ProductName VARCHAR(50)
```

```
);
```

```
CREATE TABLE Orders(
```

```
    OrderID INT PRIMARY KEY,
```

```
    CustomerID INT,
```

```
    FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)
```

```
);
```

```
CREATE TABLE OrderDetails(
```

```
    OrderID INT,
```

```
    ProductID INT,
```

```
    Quantity INT,
```

```
PRIMARY KEY(OrderID,ProductID),  
FOREIGN KEY(OrderID) REFERENCES Orders(OrderID),  
FOREIGN KEY(ProductID) REFERENCES Product(ProductID)  
);
```

OUTPUT

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)  
  
CREATE TABLE Customer( CustomerID INT PRIMARY KEY, CustomerName VARCHAR(50) );  
[ Edit inline ][ Edit ][ Create PHP code ]  
  
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)  
  
CREATE TABLE Product( ProductID INT PRIMARY KEY, ProductName VARCHAR(50) );  
[ Edit inline ][ Edit ][ Create PHP code ]  
  
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)  
  
CREATE TABLE Orders( OrderID INT PRIMARY KEY, CustomerID INT, FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID) );  
[ Edit inline ][ Edit ][ Create PHP code ]  
  
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)  
  
CREATE TABLE OrderDetails( OrderID INT, ProductID INT, Quantity INT, PRIMARY KEY(OrderID,ProductID), FOREIGN KEY(OrderID) REFERENCES Orders(OrderID), FOREIGN KEY(ProductID) REFERENCES Product(ProductID) );  
[ Edit inline ][ Edit ][ Create PHP code ]
```

Result: The database is normalized to **3NF**, eliminating redundant data.

Q6. Customers and Orders

Scenario Explanation

A company wants to identify frequent customers based on the number of orders placed. Joins combine customer and order data, while GROUP BY and HAVING identify customers with multiple orders.

QUERY

```
CREATE TABLE Customer(  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(50)  
);
```

```
CREATE TABLE Orders(  
    OrderID INT PRIMARY KEY,
```

```
OrderID INT PRIMARY KEY,  
CustomerID INT,  
FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)  
);
```

```
INSERT INTO Customer VALUES
```

```
(1,'Rahul'),
```

```
(2,'Priya'),
```

```
(3,'Amit');
```

```
INSERT INTO Orders VALUES
```

```
(101,1),
```

```
(102,1),
```

```
(103,1),
```

```
(104,2),
```

```
(105,3),
```

```
(106,3),
```

```
(107,3);
```

```
SELECT c.CustomerID,c.CustomerName,
```

```
COUNT(o.OrderID) AS TotalOrders
```

```
FROM Customer c
```

```
JOIN Orders o
```

```
ON c.CustomerID=o.CustomerID
```

```
GROUP BY c.CustomerID,c.CustomerName
```

```
HAVING COUNT(o.OrderID)>=3;
```

OUTPUT

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
CREATE TABLE Customer( CustomerID INT PRIMARY KEY, CustomerName VARCHAR(50) );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
CREATE TABLE Orders( OrderID INT PRIMARY KEY, CustomerID INT, FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID) );
```

[Edit inline] [Edit] [Create PHP code]

✓ 3 rows inserted. (Query took 0.0011 seconds.)

```
INSERT INTO Customer VALUES (1, 'Rahul'), (2, 'Priya'), (3, 'Amit');
```

[Edit inline] [Edit] [Create PHP code]

✓ 7 rows inserted. (Query took 0.0011 seconds.)

```
INSERT INTO Orders VALUES (101,1), (102,1), (103,1), (104,2), (105,3), (106,3), (107,3);
```

[Edit inline] [Edit] [Create PHP code]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✓ Showing rows 0 - 1 (2 total, Query took 0.0006 seconds.)

```
SELECT c.CustomerID, c.CustomerName, COUNT(o.OrderID) AS TotalOrders FROM Customer c JOIN Orders o ON c.CustomerID=o.CustomerID GROUP BY c.CustomerID, c.CustomerName HAVING COUNT(o.OrderID) >= 3;
```

✓ 7 rows inserted. (Query took 0.0011 seconds.)

```
INSERT INTO Orders VALUES (101,1), (102,1), (103,1), (104,2), (105,3), (106,3), (107,3);
```

[Edit inline] [Edit] [Create PHP code]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

✓ Showing rows 0 - 1 (2 total, Query took 0.0006 seconds.)

```
SELECT c.CustomerID, c.CustomerName, COUNT(o.OrderID) AS TotalOrders FROM Customer c JOIN Orders o ON c.CustomerID=o.CustomerID GROUP BY c.CustomerID, c.CustomerName HAVING COUNT(o.OrderID) >= 3;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

CustomerID	CustomerName	TotalOrders
1	Rahul	3
3	Amit	3

Q7. Multi-Valued Dependencies (4NF)

Scenario Explanation

A student can have multiple hobbies and multiple languages. Storing both in one table creates multi-valued dependencies. The solution is to decompose the relation into 4NF.

QUERY

CREATE TABLE StudentHobby(

StudentID INT,

Hobby VARCHAR(50),

```
PRIMARY KEY(StudentID,Hobby)
);
```

```
CREATE TABLE StudentLanguage(
    StudentID INT,
    Language VARCHAR(50),
    PRIMARY KEY(StudentID,Language)
);
```

OUTPUT

 MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)

```
CREATE TABLE StudentHobby( StudentID INT, Hobby VARCHAR(50), PRIMARY KEY(StudentID,Hobby) );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

 MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
CREATE TABLE StudentLanguage( StudentID INT, Language VARCHAR(50), PRIMARY KEY(StudentID,Language) );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Result: The database satisfies **4NF**.

Q8. Integrity Constraints

Scenario Explanation

Improper constraints allow invalid or duplicate data. Primary keys, foreign keys, unique constraints, check constraints, and NOT NULL constraints maintain data integrity.

QUERY

```
CREATE TABLE Department(
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);
```

```
CREATE TABLE Employee(
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50) NOT NULL,
    Salary DECIMAL(10,2) CHECK(Salary>0),
    Email VARCHAR(100) UNIQUE,
    DeptID INT,
    FOREIGN KEY(DeptID)
    REFERENCES Department(DeptID)
);
```

OUTPUT

```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)

CREATE TABLE Department( DeptID INT PRIMARY KEY, DeptName VARCHAR(50) );
[ Edit inline ] [ Edit ] [ Create PHP code ]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

CREATE TABLE Employee( EmpID INT PRIMARY KEY, EmpName VARCHAR(50) NOT NULL, Salary DECIMAL(10,2) CHECK(Salary>0), Email VARCHAR(100) UNIQUE,
DeptID INT, FOREIGN KEY(DeptID) REFERENCES Department(DeptID) );
[ Edit inline ] [ Edit ] [ Create PHP code ]

```

Q9. Join Dependency (5NF)

Scenario Explanation

A relation containing supplier, part, and project information has join dependencies that create redundancy. Decompose it into smaller tables to satisfy 5NF.

SQL Solution

```
CREATE TABLE SupplierPart(
    SupplierID INT,
    PartID INT,
    PRIMARY KEY(SupplierID,PartID)
);
```

```
CREATE TABLE SupplierProject(
```

```
SupplierID INT,  
ProjectID INT,  
PRIMARY KEY(SupplierID,ProjectID)  
);
```

```
CREATE TABLE PartProject(  
    PartID INT,  
    ProjectID INT,  
    PRIMARY KEY(PartID,ProjectID)  
);
```

OUTPUT

 MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

```
CREATE TABLE SupplierPart( SupplierID INT, PartID INT, PRIMARY KEY(SupplierID,PartID) );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

 MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

```
CREATE TABLE SupplierProject( SupplierID INT, ProjectID INT, PRIMARY KEY(SupplierID,ProjectID) );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

 MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

```
CREATE TABLE PartProject( PartID INT, ProjectID INT, PRIMARY KEY(PartID,ProjectID) );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Result: The schema satisfies **5NF**.

Q10. SQL Query Optimization

Scenario Explanation

A system retrieves employee and department data using inefficient queries. Optimizing with INNER JOIN and indexes improves execution speed and reduces query cost.

SQL Solution

```
CREATE TABLE Department(  
    DeptID INT PRIMARY KEY,
```

```
DeptName VARCHAR(50)
);
```

```
CREATE TABLE Employee(
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT,
    FOREIGN KEY(DeptID)
    REFERENCES Department(DeptID)
);
```

```
CREATE INDEX idx_dept
ON Employee(DeptID);
```

```
SELECT e.EmpName,d.DeptName
FROM Employee e
INNER JOIN Department d
ON e.DeptID=d.DeptID;
```

OUTPUT

The screenshot displays a MySQL query execution interface with four queries and their results. Each query is preceded by a green checkmark and a status message: "MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)".

- Query 1:** `CREATE TABLE Departmentss( DeptID INT PRIMARY KEY, DeptName VARCHAR(50) );`
- Query 2:** `CREATE TABLE Employees( EmpID INT PRIMARY KEY, EmpName VARCHAR(50), DeptID INT, FOREIGN KEY(DeptID) REFERENCES Department(DeptID) );`
- Query 3:** `CREATE INDEX idx_dept ON Employee(DeptID);`
- Query 4:** `SELECT e.EmpName,d.DeptName FROM Employees e INNER JOIN Departmentss d ON e.DeptID=d.DeptID;`

Below the queries, there are links for "Edit inline", "Edit", and "Create PHP code". At the bottom, there is a table header with columns "EmpName" and "DeptName".

Result: The optimized query executes faster due to proper joins and indexing.